COMPUTER NETWORKS
ASSIGNMENT 1 — QUESTIONS AND ANSWERS

Total: 50 Marks | CO1

Student details

Name: Abdul Raashid S Register Number: 1911260116

GitHub Repository Link: https://github.com/Abdul999-hub/computer-networks

Note: The answers follow the supplied assignment. Where the question does not provide a required value (for example, AGV traffic per vehicle), the assumption is stated clearly. Practical screenshots, Packet Tracer files, Wireshark captures and actual local command outputs must be added by the student.

TASK 1 — Size the Traffic Before Choosing Anything

Question: Calculate the traffic generated by one production line and by eight lines; identify the highest-volume and most delay-sensitive sources; assess a 1 Gbps uplink and the effect of a shared CCTV queue; rank the traffic types by the harm caused by 100 ms delay. Then recalculate if CCTV records locally and sends clips only 5% of the time.

A. Robot controllers: each controller sends 200 bytes every 2 ms. Traffic per controller = (200 × 8) / 0.002 = 800,000 bit/s = 0.8 Mbps. For 40 controllers: 40 × 0.8 = 32 Mbps.

B. Vibration sensors: 600 × 50 bytes/s × 8 = 240,000 bit/s = 0.24 Mbps.

C. CCTV: 24 × 4 Mbps = 96 Mbps.

D. Office users: 10% of 150 = 15 active users. Traffic = 15 × 2 Mbps = 30 Mbps.

Traffic source	**Traffic per line**
Robot controllers	32 Mbps
Vibration sensors	0.24 Mbps
CCTV cameras	96 Mbps
Office users	30 Mbps
TOTAL	158.24 Mbps

For eight lines: 158.24 × 8 = 1,265.92 Mbps = approximately 1.266 Gbps.

CCTV sends the most data (96 Mbps). Robot controllers have the strictest delay requirement because their commands must arrive within 10 ms. Therefore, the highest-volume source and the most delay-sensitive source are not the same.

A 1 Gbps uplink is sufficient for one line (158.24 Mbps), but not for eight lines (1,265.92 Mbps). The eight-line load exceeds 1 Gbps by 265.92 Mbps, even before allowing for protocol overhead and growth. If CCTV and robot commands share an ordinary FIFO queue, large video packets can wait ahead of small control packets, increasing command delay. Separate VLANs/queues and QoS priority for control traffic are required.

Ranking by the harm caused by a 100 ms delay

1. Robot commands — 100 ms is ten times the stated 10 ms deadline and can disrupt control.

2. Vibration readings — delayed readings reduce the usefulness of time-sensitive condition monitoring.
3. CCTV — delay causes lag in live viewing, although local recording can preserve the footage.
4. Office ERP/e-mail — users experience slower response, but these applications are generally less time-critical.

TASK 2 — Count the Domains in the Contractor's Drawing

Question: For three hubs with eight PCs each, connected to one 24-port switch, then a router and a second switch with 12 IP phones, calculate collision and broadcast domains; identify OSI layers, point-to-point/multipoint links and standards bodies. Recalculate after replacing the hubs with switches.

Collision domains: each hub and all devices attached to it share one collision domain, giving 3. The first switch-to-router link is one domain; the router-to-second-switch link is one domain; and the 12 phone ports on the second switch are 12 separate domains. Total = 3 + 1 + 1 + 12 = 17 collision domains.

Broadcast domains: a router separates broadcast domains. The two router interfaces connect two separate LANs, so there are 2 broadcast domains (assuming no additional VLANs).

Device / technology	OSI layer	Reason
Hub	Layer 1 — Physical	Repeats incoming electrical signals to all ports; does not filter frames.
Switch	Layer 2 — Data Link	Learns MAC addresses and forwards Ethernet frames.
Router	Layer 3 — Network	Forwards packets using IP addresses and separates broadcast domains.
IP phone	End device / application endpoint	Uses network layers to carry voice; it is not a Layer 2/3 forwarding device in this drawing.

A hub cannot stop collisions because all connected stations share the same collision domain and the hub repeats signals without buffering or making forwarding decisions.

Point-to-point connections include each hub-to-switch link, switch-to-router link, router-to-second-switch link and each phone-to-switch link. The hub segment is logically multipoint because several PCs share the same medium/collision domain.

Standard / specification	Defining body
Ethernet — IEEE 802.3	IEEE
Wi-Fi — IEEE 802.11	IEEE
IPv4 — RFC 791	IETF
OSI model — ISO 7498	ISO
5G radio specifications / IMT-2020 framework	3GPP develops cellular specifications; ITU defines the IMT framework/requirements

Standard-compliant equipment improves interoperability, compatibility, predictable operation, security support and future expansion.

TASK 3 — Read a Captured Frame Layer by Layer

Question: Identify the OSI layer and PDU for each captured line; calculate routers crossed from TTL 64 to 61; calculate bytes and overhead for a 12-byte command and a 1,400-byte packet; map faults to layers; map TCP/IP to OSI. Complete the encapsulation code and analyse direct Ethernet transport.

Captured field	OSI layer	PDU
Ethernet II	Layer 2 — Data Link	Frame
IPv4	Layer 3 — Network	Packet
TCP	Layer 4 — Transport	Segment
Modbus command data	Application layer	Data

Routers crossed = initial TTL − received TTL = 64 − 61 = 3 routers, assuming each router decrements TTL once.

For the 12-byte command: Ethernet header 14 + IPv4 header 20 + TCP header 20 + data 12 + FCS 4 = 70 bytes. Overhead = 14 + 20 + 20 + 4 = 58 bytes. Overhead percentage = (58 / 70) × 100 = 82.86%.

For a 1,400-byte payload: total = 1,400 + 58 = 1,458 bytes. Overhead percentage = (58 / 1,458) × 100 = 3.98%. Thus, fixed headers consume a much larger proportion of a small command than of a large data packet. These calculations include the Ethernet header and FCS, but not preamble, inter-frame gap or VLAN tag.

Problem	Responsible layer / function
Wrong destination MAC	Layer 2 — Ethernet addressing and switching
TTL reaches zero	Layer 3 — IP forwarding; router discards packet and normally sends ICMP Time Exceeded
Destination TCP port closed	Layer 4 — TCP/UDP transport service; TCP may return a reset
TCP/IP model	**OSI layers**
Application	Layers 7, 6 and 5 — Application, Presentation, Session
Transport	Layer 4 — Transport
Internet	Layer 3 — Network
Network Access / Link	Layers 2 and 1 — Data Link and Physical

Complete the Code — tested Python 3 program

```
LAYERS = [("Application", "HTTP"), ("Transport", "TCP"),
          ("Network", "IP"), ("Data link", "ETH")]

def send(message):
    pdu = message
    for layer, tag in LAYERS:
        if tag == "ETH":
            pdu = "[" + tag + "|" + pdu + "|FCS]"
        else:
            pdu = "[" + tag + "|" + pdu + "]"
        print(f"{layer} adds {tag} -> {len(pdu)} bytes")
    return pdu

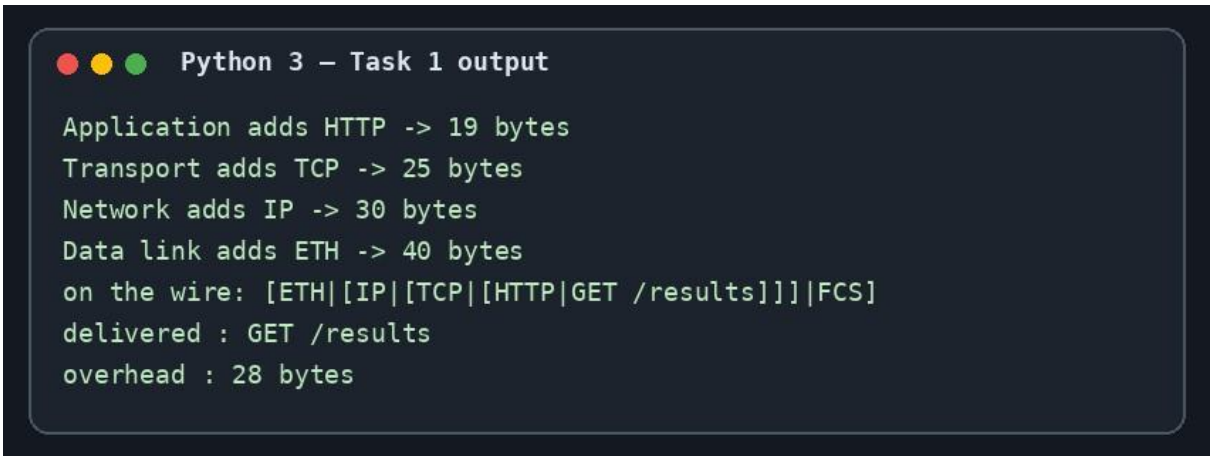
def receive(frame):
    pdu = frame
    for layer, tag in reversed(LAYERS):
        head = "[" + tag + "|"
        tail = "|FCS]" if tag == "ETH" else "]"
        assert pdu.startswith(head) and pdu.endswith(tail), f"{layer}: bad header"
        pdu = pdu[len(head):-len(tail)]
    return pdu

frame = send("GET /results")
print("on the wire:", frame)
```

```
print("delivered :", receive(frame))
print(f"overhead : {len(frame) - len('GET /results')} bytes")
```

Expected execution result:

```
Application adds HTTP -> 19 bytes
Transport adds TCP -> 25 bytes
Network adds IP -> 30 bytes
Data link adds ETH -> 40 bytes
on the wire: [ETH|[IP|[TCP|[HTTP|GET /results]]]|FCS]
delivered : GET /results
overhead : 28 bytes
```



Output screenshot 1: captured from actual Python 3 execution of the code shown above.

The supplied code's wrapper is a teaching illustration: its character-count overhead (28 bytes) is not the same as the real Ethernet/IP/TCP header calculation (58 bytes) above.

TASK 4 — Choose a Medium for Each Run

Question: Select a medium for the 85 m welding-area robot cell, 140 m office-to-shop-floor run, 550 m shop-floor-to-data-centre run and 400 km plant-to-head-office run; identify the copper-distance trap; calculate Shannon capacities at 40 dB and 15 dB; complete Nyquist/Shannon code; calculate the SNR needed for 56 kbps over 3,100 Hz; assess Wi-Fi 6 at 15 dB.

Run	Recommended medium	Reason
Robot cell near welders, 85 m	Industrial shielded Cat6A S/FTP (or fibre where electrical isolation is needed)	Within 100 m copper Ethernet channel limit; shielding and proper bonding help reduce EMI.
Office to shop-floor switch, 140 m	Fibre optic	Exceeds the usual 100 m twisted-pair Ethernet channel limit.
Shop floor to data centre, 550 m	Fibre optic; multimode may suit a short campus run, subject to transceiver reach	Copper Ethernet is beyond its usual 100 m limit; fibre is immune to EMI.
Plant to head office, 400 km	Single-mode fibre leased service	Suitable for long-distance, high-capacity communication.

The 400 km run cannot use ordinary copper Ethernet. In addition, the 140 m and 550 m runs exceed the typical 100 m copper Ethernet channel limit.

Shannon formula: $C = B \log_2(1 + SNR)$, where SNR is a linear ratio. For 40 dB, SNR = 10^(40/10) = 10,000. $C = 250 \times 10^6 \times \log_2(10{,}001)$ = approximately 3,321.93 Mbps (about 3.322 Gbps).

For 15 dB, SNR = 10^(15/10) = 31.6228. $C = 250 \times 10^6 \times \log_2(1 + 31.6228)$ = approximately 1,256.9 Mbps. The lower SNR reduces theoretical capacity and indicates a noisier link. Shannon capacity is an upper bound, not a guaranteed Ethernet speed.

Actual link speed is lower because of modulation/coding limits, protocol overhead, implementation limits, interference, attenuation, retransmissions and hardware constraints.

Complete the Code — tested Python 3 program

```
import math

def nyquist(bandwidth_hz, levels):
    return 2 * bandwidth_hz * math.log2(levels)

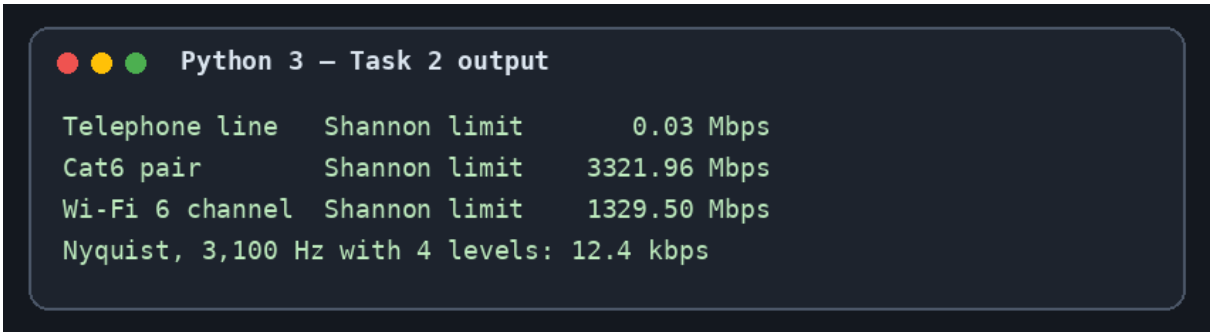
def shannon(bandwidth_hz, snr_db):
    snr = 10 ** (snr_db / 10)
    return bandwidth_hz * math.log2(1 + snr)

media = [("Telephone line", 3100, 30),
         ("Cat6 pair", 250e6, 40),
         ("Wi-Fi 6 channel", 160e6, 25)]

for name, bw, snr in media:
    print(f"{name:<16} Shannon limit {shannon(bw, snr) / 1e6:10.2f} Mbps")
print(f"Nyquist, 3,100 Hz with 4 levels: {nyquist(3100, 4) / 1e3:.1f} kbps")
```

Expected execution result:

```
Telephone line   Shannon limit      0.03 Mbps
Cat6 pair        Shannon limit   3321.93 Mbps
Wi-Fi 6 channel  Shannon limit   1329.50 Mbps
Nyquist, 3,100 Hz with 4 levels: 12.4 kbps
```



Output screenshot 2: captured from actual Python 3 execution of the code shown above.

For 56 kbps over 3,100 Hz: $56{,}000 = 3{,}100 \log_2(1 + SNR)$. Thus SNR = 2^(56,000/3,100) − 1 ≈ 274,132, which is approximately 54.38 dB. This is an extremely demanding SNR for an ordinary telephone line.

Wi-Fi 6 at 160 MHz and 15 dB: SNR = 31.6228. Shannon capacity = $160 \times 10^6 \times \log_2(32.6228) \approx 804.4$ Mbps. The theoretical capacity is not a latency guarantee. In a welding area, interference, contention and variable retransmissions make Wi-Fi unsuitable as the sole critical robot-control medium. A properly engineered industrial wireless system may support monitoring or non-critical traffic.

TASK 5 — Wi-Fi 6 or 5G for Moving Equipment?

Question: For an AGV travelling 1.2 km at 2 m/s, compare Wi-Fi APs 30 m apart with 150 ms handover and 5G cells 300 m across with 20 ms handover; map 5G service types; identify uses for Bluetooth, Zigbee and LoRa; assess 802.11r; test whether 600 sensors fit a 5 kbps LoRa gateway.

Assume the route begins in the first coverage area and ends in the last, with no extra handover at the start or finish.

Wi-Fi: 1,200 / 30 = 40 coverage sections, so approximately 39 handovers. Total interruption = 39 × 150 ms = 5,850 ms = 5.85 s.

5G: 1,200 / 300 = 4 coverage sections, so approximately 3 handovers. Total interruption = 3 × 20 ms = 60 ms = 0.06 s. This is much less than Wi-Fi, but an individual 20 ms handover still exceeds the 10 ms command deadline. The given 5G handover figure alone therefore does not prove that the deadline can be met.

Traffic / device	5G service type	Reason
AGVs	URLLC	Designed for very low latency and high reliability.
CCTV cameras	eMBB	High data rate for continuous video.
Large population of sensors	mMTC	Supports many low-rate connected devices.

Bluetooth is suitable for short-range peripherals and setup/maintenance. Zigbee suits low-power, low-rate sensor networks. LoRa/LoRaWAN suits long-range, very low-data-rate telemetry, not time-critical robot control.

802.11r fast transition reduces Wi-Fi roaming authentication delay, but does not remove radio coverage gaps, interference, contention, packet loss or unpredictable latency. It cannot by itself guarantee a 10 ms deadline.

TASK 6 — Interpret an Ethernet Port Report

Question: Explain late collisions and CRC errors; identify the likely duplex mismatch and give a correction; interpret the supplied MAC addresses; assess jumbo frames; compare industrial ring and star recovery.

A late collision is a collision detected outside the normal collision window on a half-duplex Ethernet link. It can indicate a duplex mismatch, excessive cable length or a physical-layer fault. A CRC error means the received frame failed its frame check sequence and was likely corrupted in transit.

The combination of late collisions and CRC errors, with the PC forced to 100 Mb/s half duplex while the switch port is auto-negotiating, strongly suggests a duplex mismatch. A common correction is to enable auto-negotiation at both ends. On Linux, for interface eth0: `sudo ethtool -s eth0 autoneg on`.

01:00:5e:0a:01:02 is an IPv4 multicast Ethernet MAC address (the 01:00:5e range is used for IPv4 multicast mapping), so it represents a group, not one individual unicast device. The first three bytes of 3c:52:82:4e:11:07 are the OUI portion (3C:52:82), which identifies the registered manufacturer; the exact manufacturer should be confirmed using an IEEE OUI lookup rather than guessed.

Jumbo frames (often around 9,000-byte MTU) may improve large CCTV storage transfers by reducing per-byte packet overhead and packet-processing load. They help only if every device on the path (sender NIC, switches, receiver NIC/server and any routed interfaces) supports and is configured for a compatible MTU. Jumbo frames do not fix a slow disk or insufficient link capacity.

TASK 7 — Will the Receiver Notice the Error?

Question: Find the CRC-3 remainder for data 1101011011 using generator 1011; test received frame 1101011011101; state burst-error guarantees and an undetectable pattern; show two-dimensional parity and an undetected four-bit pattern; calculate the Internet checksum; complete the CRC code and compare CRC with even parity.

Generator 1011 has degree 3. Append three zero bits to the data: 1101011011000. Modulo-2 division by 1011 gives remainder 100. Therefore, the transmitted codeword is 1101011011100.

For the received frame 1101011011101, division by 1011 gives a non-zero remainder (001), so the receiver rejects the frame.

A degree-3 generator detects every burst error of length 3 bits or less. A possible undetected error pattern is 1011 itself (aligned as an error polynomial), because it is exactly the generator polynomial and therefore divides evenly by it. More generally, any error polynomial that is a multiple of the generator is undetected.

Two-dimensional even parity example

Data bit 1	Data bit 2	Data bit 3	Data bit 4	Row parity
1	0	1	1	1
1	1	0	0	0
0	1	1	0	0
1	0	0	1	0
1	0	0	0	1

The final column makes each row even; the final row makes each column even. A rectangular four-bit error, for example flipping the four corner data bits of a 4 × 4 data block, leaves each affected row and column with two errors, so all row and column parity checks still pass. This is an example of an error pattern that two-dimensional parity misses.

Internet checksum

Add the four 16-bit words: 0x4500 + 0x0073 + 0x0000 + 0x4000 = 0x8573. The 1's complement is 0xFFFF − 0x8573 = 0x7A8C. Internet checksum = 7A8C.

Complete the Code — tested Python 3 program

```
def mod2_div(dividend, divisor):
    rem = list(dividend)
    for i in range(len(dividend) - len(divisor) + 1):
        if rem[i] == "1":
            for j in range(len(divisor)):
                rem[i + j] = str(int(rem[i + j]) ^ int(divisor[j]))
    return "".join(rem[-(len(divisor) - 1):])

def encode(data, gen):
    crc = mod2_div(data + "0" * (len(gen) - 1), gen)
    return data + crc

def check(frame, gen):
    return set(mod2_div(frame, gen)) == {"0"}

GEN = "1011"
frame = encode("1101011011", GEN)
print("transmitted:", frame)
```

```
print("no error   :", check(frame, GEN))
bad = frame[:4] + ("1" if frame[4] == "0" else "0") + frame[5:]
print("1-bit error:", check(bad, GEN))
burst = frame[:2] + "".join("1" if b == "0" else "0" for b in frame[2:5]) + frame[5:]
print("3-bit burst:", check(burst, GEN))
```

Expected execution result:



Output screenshot 3: captured from actual Python 3 execution of the code shown above.

```
transmitted: 1101011011100
no error   : True
1-bit error: False
3-bit burst: False
```

TASK 8 — Correct the Error or Ask Again?

Question: Calculate Hamming check bits for 16 data bits and overhead; find the minimum distance of the given code; correct the received Hamming(7,4) word; compare FEC and ARQ; complete the decoder and analyse a two-bit error.

For m = 16, choose r such that 2^r ≥ m + r + 1. r = 4 gives 16 ≥ 21 (false). r = 5 gives 32 ≥ 22 (true). Therefore, 5 check bits are required and the codeword length is 21 bits. Check-bit overhead relative to the 16 data bits = (5/16) × 100 = 31.25%.

For codewords 00000, 01011, 10101 and 11110, the minimum pairwise Hamming distance is 3. Thus dmin = 3. Guaranteed detection = dmin − 1 = 2 errors; guaranteed correction = floor((dmin − 1)/2) = 1 error.

The original codeword is 0110011 and the received word is 0110111. The syndrome identifies bit position 5. Flip bit 5 to restore 0110011. Data positions 3, 5, 6 and 7 give 1011.

FEC adds redundant bits so the receiver can correct certain errors without requesting another transmission. ARQ detects an error and retransmits, adding delay. FEC is useful for real-time video and cellular links where waiting for retransmission may be undesirable; ARQ is useful when reliable retransmission is acceptable.

Complete the Code — tested Python 3 program

```
def encode(d):
    p1 = d[0] ^ d[1] ^ d[3]
    p2 = d[0] ^ d[2] ^ d[3]
    p3 = d[1] ^ d[2] ^ d[3]
    return [p1, p2, d[0], p3, d[1], d[2], d[3]]

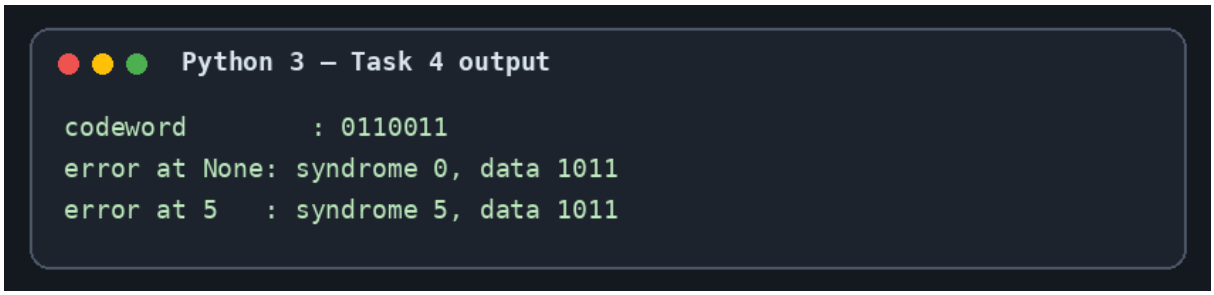
def decode(c):
```

```
        s1 = c[0] ^ c[2] ^ c[4] ^ c[6]
        s2 = c[1] ^ c[2] ^ c[5] ^ c[6]
        s3 = c[3] ^ c[4] ^ c[5] ^ c[6]
        pos = s1 + 2*s2 + 4*s3
        if pos != 0:
            c[pos - 1] ^= 1
        data = [c[2], c[4], c[5], c[6]]
        return pos, data

    word = encode([1, 0, 1, 1])
    print("codeword        :", "".join(map(str, word)))
    for flip in (None, 5):
        rx = word[:]
        if flip:
            rx[flip - 1] ^= 1
        pos, data = decode(rx)
        print(f"error at {str(flip):<4}: syndrome {pos}, data {''.join(map(str, data))}")
```

Expected execution result:

```
codeword        : 0110011
error at None: syndrome 0, data 1011
error at 5   : syndrome 5, data 1011
```



Output screenshot 4: captured from actual Python 3 execution of the code shown above.

If two bits are flipped, a standard Hamming(7,4) decoder is not guaranteed to correct the word. It may interpret the two-bit error syndrome as a single-bit position and flip a third bit, producing a wrong codeword/data. This demonstrates why the code corrects only one error.

TASK 9 — How Big Must the Window Be?

Question: For a 400 km, 1 Gbps fibre link carrying 1,500-byte frames, calculate transmission and propagation times, a, stop-and-wait utilisation and throughput, minimum full-utilisation window, sequence bits for Go-Back-N and Selective Repeat, compare retransmissions, explain HDLC/PPP framing, complete code and analyse a 64-frame Selective Repeat window.

Frame size = 1,500 × 8 = 12,000 bits. Transmission time = 12,000 / 10^9 = 0.000012 s = 12 µs. One-way propagation time = 400,000 / (2 × 10^8) = 0.002 s = 2 ms. a = propagation / transmission = 0.002 / 0.000012 = 166.67.

Stop-and-wait utilisation U = 1/(1 + 2a) = 1/(1 + 333.33) ≈ 0.00299 = 0.299%. Throughput ≈ 1 Gbps × 0.00299 = 2.99 Mbps.

For continuous full utilisation, W ≥ 1 + 2a = 334.33, so the smallest whole-number window is 335 frames.

Go-Back-N requires $W \leq 2^k - 1$. For $W = 335$, $k = 9$ gives 511, so 9 sequence-number bits are sufficient. Selective Repeat requires $W \leq 2^{(k-1)}$. With $k = 10$, the maximum safe window is 512, so 10 bits are required.

Selective Repeat resends only the missing/damaged frame(s), while Go-Back-N may resend the lost frame and all later unacknowledged frames. Selective Repeat costs more receiver memory and logic because it buffers out-of-order frames.

HDLC and PPP use the flag bit pattern 01111110 to mark frame boundaries. Bit stuffing (in HDLC) prevents the flag pattern from appearing as data.

Complete the Code — tested Python 3 program

```
def utilisation(frame_bits, rate_bps, distance_m, window, speed=2e8):
    tx = frame_bits / rate_bps
    prop = distance_m / speed
    a = prop / tx
    u = min(1, window / (1 + 2*a))
    return u, a

links = [("Campus LAN, 200 m, 1 Gbps", 1e9, 200),
         ("City fibre, 50 km, 1 Gbps", 1e9, 50000),
         ("Satellite, 36,000 km, 10 Mbps", 10e6, 36000000)]
for name, rate, dist in links:
    u1, a = utilisation(12000, rate, dist, 1)
    u7, _ = utilisation(12000, rate, dist, 7)
    u127, _ = utilisation(12000, rate, dist, 127)
    print(f"{name:<31} a = {a:7.2f} stop-and-wait {u1:6.1%} "
          f"W=7 {u7:6.1%} W=127 {u127:6.1%}")
```

Expected output (rounding as in the assignment):

```
Campus LAN, 200 m, 1 Gbps       a =    0.08 stop-and-wait  85.7% W=7 100.0% W=127 100.0%
City fibre, 50 km, 1 Gbps        a =   20.83 stop-and-wait   2.3% W=7  16.4% W=127 100.0%
Satellite, 36,000 km, 10 Mbps    a = 150.00 stop-and-wait   0.3% W=7   2.3% W=127  42.2%
```



Output screenshot 5: captured from actual Python 3 execution of the code shown above.

TASK 10 — Find Where the Delay Comes From

Question: Calculate propagation delay for 400 km of fibre and its share of measured delay; explain the 38.2 ms outlier at hop 3 and stars at hop 5; identify the largest consistent delay increase; describe own traceroute, ping and Wireshark evidence; compare SNMP/NetFlow with occasional tests.

One-way propagation delay = 400,000 / (2 × 10^8) = 2 ms. Round-trip propagation component ≈ 4 ms. Compared with the final measured RTT of 12.6 ms, this is 4/12.6 × 100 ≈ 31.75%. The remaining time includes router processing, queuing, access-network and other path delays. This comparison assumes the 400 km is the one-way path length.

Hop 3 has two values near 4 ms and one outlier at 38.2 ms. A single outlier may result from temporary queuing, congestion, control-plane processing or ICMP rate limiting; it does not prove that forwarding traffic is delayed by the same amount. Hop 5 stars mean no probe response was received before timeout, possibly due to filtering or rate limiting. Since hop 6 responds, hop 5 is not necessarily a forwarding failure.

Using the representative values, the largest consistent increase is between hop 3 (about 4.0 ms) and hop 4 (about 11.8 ms): approximately 7.8 ms. Traceroute RTTs are cumulative probe round-trip times, not direct one-way per-link delays, so this is an approximate hop-to-hop indication.

For the required personal evidence, run `tracert <destination>` on Windows or `traceroute <destination>` on Linux, run `ping -n 10 <destination>` (Windows) or `ping -c 10 <destination>` (Linux), and capture a real connection in Wireshark. In the report, include screenshots and label Ethernet (Layer 2), IP (Layer 3), TCP/UDP (Layer 4) and application data. Actual results cannot be supplied from the assignment sheet and must be collected on the student's device.

Method	What it shows	Typical detection value
SNMP counters	Interface errors, drops, utilisation and device health	Continuous/polling-based trend and interface fault visibility
NetFlow / flow records	Source/destination, ports, protocol and traffic volume	Identifies heavy flows and traffic changes; not packet-by-packet latency by itself
Ping	Reachability, RTT and packet loss to a target	Simple end-to-end symptoms when tests run
Traceroute	Path and responding hop RTTs	Helps localise path changes or unusual hop response; results can be affected by ICMP handling

Continuous SNMP/flow monitoring can show when counters or traffic patterns change and can retain history. Ping/traceroute are useful diagnostic snapshots but may miss intermittent events between tests.

TASK 11 — Defend the Network Design

Question: Recommend media, wireless technologies and Ethernet settings for each plant area; choose error/flow control methods with calculations; identify two major risks; assess five plants; compare with an all-private-5G plant.

Area / application	Recommended design	Reason / settings
Stationary robot cells	Industrial Ethernet over shielded Cat6A S/FTP within 100 m, or fibre where EMI isolation is needed	Full duplex, matched speed/duplex, QoS priority and industrial deterministic control.
AGVs / mobile equipment	Private 5G URLLC with overlapping coverage and local edge control	Mobility and low-latency service; handover must be tested against 10 ms deadline.
Office users	Wi-Fi 6	General user access; separate office VLAN and appropriate security.

CCTV	Wired Gigabit Ethernet to access switches and fibre backbone	Predictable high-volume video; local recording can reduce backbone load.
Sensors	Industrial Ethernet for critical sensors; Zigbee/LoRa/mMTC for suitable low-rate telemetry	Select based on rate, range, battery and latency needs.
Backbone / data centre	Fibre optic	High capacity and immunity to electromagnetic interference.
Head-office WAN	Single-mode fibre leased service	Long-distance capacity and reliability.

Error and flow control: Ethernet FCS/CRC detects frame corruption on Ethernet links. TCP provides sequencing, acknowledgements, retransmission and receiver flow control for office/ERP traffic. For the long-distance WAN, sliding-window ARQ is required rather than stop-and-wait: Task 9 gives only about 2.99 Mbps for stop-and-wait, while a 64-frame Selective Repeat window gives about 19.14% utilisation or 191.4 Mbps under the stated simplified model. Critical robot/AGV traffic should use deterministic industrial control, QoS and suitable FEC/redundancy where supported, rather than relying on ordinary TCP retransmission.

Risk 1: congestion. Eight lines produce 1.26592 Gbps, greater than a 1 Gbps uplink. Risk 2: EMI and wireless variability can cause errors, packet loss or deadline misses near welding equipment and during mobility.

For five similar plants, raw traffic = 1.26592 × 5 = 6.3296 Gbps. A 10 Gbps inter-site/core capacity is a reasonable minimum starting point based on this calculated load, with additional headroom, redundancy and traffic engineering. If local CCTV recording reduces each plant to 536.32 Mbps, five plants produce 2.6816 Gbps.

TASK 12 — Re-evaluate and Adapt the Design — Day-4 Design Challenge

Question: With private 5G added and 50 AGVs requiring that commands never miss a 10 ms deadline, identify affected media/wireless/error-control settings and recalculate traffic; test the original design using handover/delay figures; describe the adapted design; defend the decision with numbers and trade-offs.

A. Impact Analysis

The assignment does not state the traffic generated by each AGV. For a numerical estimate, assume each AGV generates the same command traffic as one robot controller: 200 bytes every 2 ms. This assumption must be confirmed with the AGV supplier or measured in the actual system.

Traffic per AGV = (200 × 8) / 0.002 = 0.8 Mbps. For 50 AGVs: 50 × 0.8 = 40 Mbps. Original eight-line traffic = 1,265.92 Mbps. New estimated total = 1,265.92 + 40 = 1,305.92 Mbps (about 1.306 Gbps). A 1 Gbps uplink is therefore insufficient. The affected choices are wireless access (private 5G for moving AGVs), coverage planning and handover, QoS, redundancy and backbone capacity. Stationary critical devices can remain on industrial Ethernet.

B. Test the Original Design

Wi-Fi: 1,200 m / 30 m = 40 coverage sections, giving approximately 39 handovers. At 150 ms each, cumulative interruption = 39 × 150 = 5,850 ms = 5.85 s.

5G: 1,200 m / 300 m = 4 coverage sections, giving approximately 3 handovers. At 20 ms each, cumulative interruption = 3 × 20 = 60 ms. 5G is much better than the stated Wi-Fi case, but a 20 ms handover is still longer

than the 10 ms command deadline. Therefore, the original design cannot be accepted as meeting the strict deadline on these figures alone.

C. Adaptation or Redesign

Modify the original design into a hybrid architecture: use private 5G URLLC for AGVs; retain industrial Ethernet for stationary robot controllers; use Wi-Fi 6 for office users; use wired Ethernet/fibre for CCTV and storage; use fibre for plant backbone and WAN; and use mMTC/other low-power technologies only for non-critical low-rate sensors.

To protect the 10 ms requirement, engineer overlapping 5G coverage, make-before-break or dual connectivity where supported, local edge control, redundant paths, QoS prioritisation and continuous latency/handover testing. Safety-critical motion control should have an appropriate local fail-safe behaviour rather than depending solely on a remote wireless command.

D. Evidence-Based Defence

The estimated AGV addition is 40 Mbps, bringing the eight-line total to 1,305.92 Mbps under the stated traffic assumption. Wi-Fi's estimated cumulative handover interruption is 5.85 s, compared with 60 ms for the stated 5G handover. However, 20 ms per 5G handover still exceeds the 10 ms deadline. The hybrid design therefore improves mobility and traffic separation, but must be validated with measured end-to-end latency, packet loss and handover performance. The decision is MODIFY, not fully replace: wired links preserve predictable high-capacity paths, while private 5G provides mobility.